TECHWIZ 7
THE WORLD TECH CHAMPIONSHIP

Aptech
Unleash your potential



Software Requirements Specification
Version 1.0

UrbanTransit IQ

Project Name: UrbanTransit IQ

Theme: TransitVerse Intelligence

Category: Data Science Intelligence Arena

Table of Contents

1.1 Background and Necessity for the Application 2

1.2 Proposed Solution 3

1.3 Purpose of the Document 30

1.4 Scope of Project 31

1.5 Constraints 32

1.6 Functional Requirements 32

1.7 Non-Functional Requirements 38

1.8 Competition Integrity and Anti-Shortcut Requirements 38

1.9 Interface Requirements 41

1.9.1 Hardware 41

1.9.2 Software 41

1.10 Project Deliverables 42

© Aptech Limited

1.1 Background and Necessity for the Application

Public transport applications generate large volumes of data through ticketing transactions, route operations, passenger counts, vehicle schedules, stop-level activity, trip timings, delays, and service records. Analyzing this information is essential for understanding passenger demand, identifying overcrowded routes, improving service reliability, reducing delays, and optimizing public transport schedules.

Traditional transport analysis often depends on periodic reports, manually prepared spreadsheets, or limited operational summaries. These approaches may not reveal complex relationships between passenger demand, route performance, travel time, occupancy, delays, stop-level usage, and schedule efficiency.

The **'UrbanTransit IQ'** application automatically analyzes large public transport datasets to identify peak travel periods, overcrowded routes, underutilized services, and delay patterns. It also identifies passenger-flow trends, route bottlenecks, and opportunities for schedule improvement. It also uses Data Science and Big Data technologies to forecast passenger demand, compare route performance, detect anomalies, and generate data-driven operational recommendations.

The goal is to develop a scalable, reliable, and intelligent transport analytics application that supports public transport operators, transport authorities, analysts, and administrators. It enables them to make better scheduling, capacity, and route-planning decisions.



© Aptech Limited

1.2 Proposed Solution

The proposed solution introduces a Big Data and Data Science-powered, Web-based public transport analytics platform that processes large volumes of ticketing, route, timing, passenger-count, vehicle, stop, and delay data.

The application collects transport data from multiple sources and processes it using the Apache Hadoop ecosystem, Apache Spark, PySpark, and Spark SQL. Raw transport records are validated, cleaned, integrated, transformed, and converted into analytical features.

The application analyzes passenger movements, boarding and alighting patterns, route occupancy, stop-level demand, travel time, delay duration, punctuality, route utilization, service frequency, peak periods, and passenger-flow patterns.

The solution must identify:

- Peak travel periods
- Overcrowded routes
- Overcrowded trips
- High-demand stops
- Low-performing routes
- Underutilized services
- Repeated delay patterns
- Delay-prone stops
- Route bottlenecks
- Passenger-flow imbalances
- Missed demand opportunities
- Schedule-adjustment opportunities

To independently validate important analytical outcomes, teams must develop two separate analytical pipelines:

1. **Big Data Pipeline:** Apache Hadoop ecosystem, HDFS, Apache Spark, PySpark, Spark SQL, and Spark MLlib
2. **Python Data Science Pipeline:** Python, Pandas, NumPy, Scikit-learn, XGBoost, Statsmodels, or other suitable Data Science libraries

© Aptech Limited

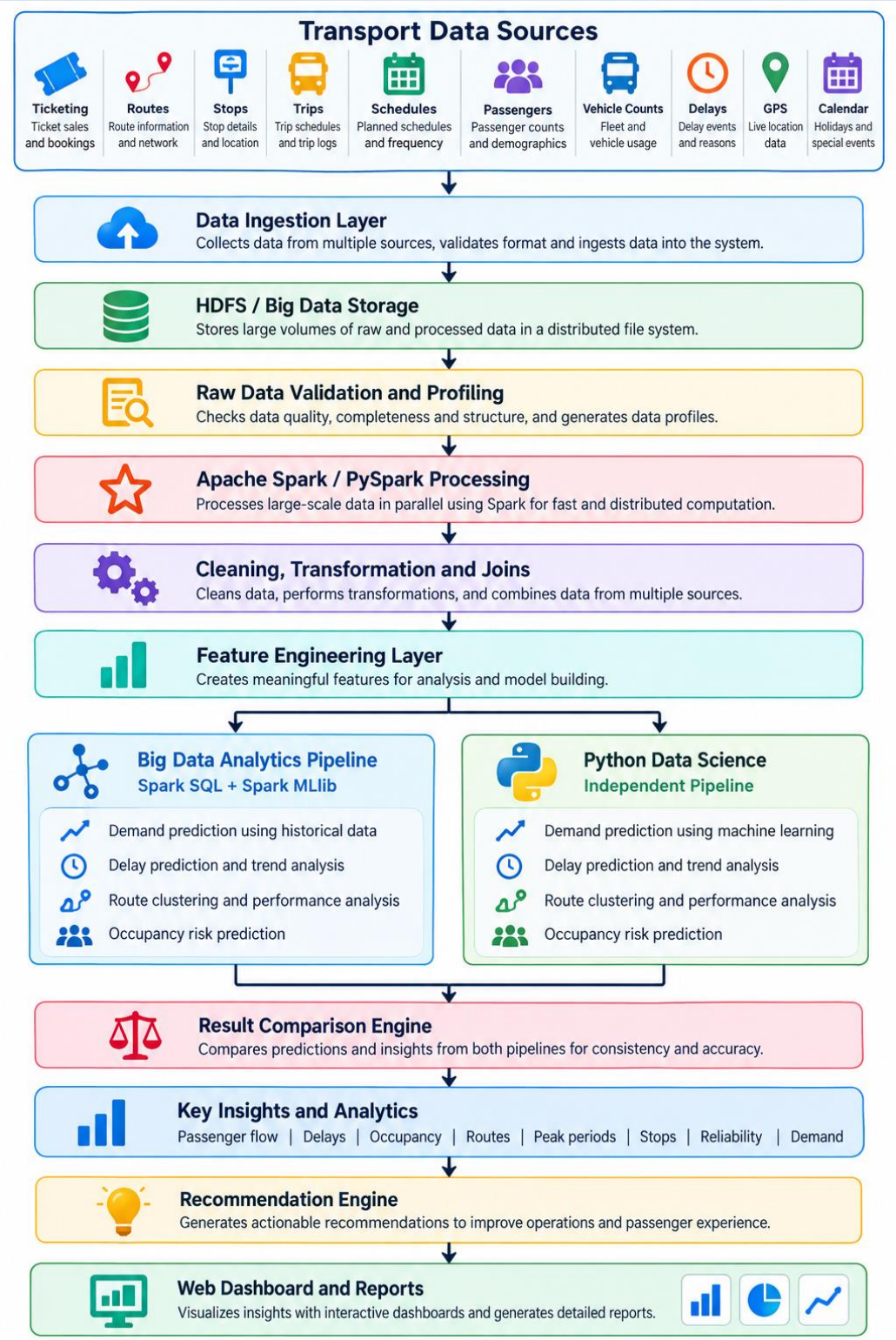
Both pipelines must independently process selected analytical problems using the same underlying dataset. Their outputs must then be compared to identify agreement, disagreement, and modelling differences.

The application must also support passenger-demand forecasting, route clustering, delay prediction, occupancy-risk prediction, anomaly detection, what-if analysis, and evidence-based schedule recommendations.



© Aptech Limited

The sample architecture for **UrbanTransit IQ** application is as follows:



Sample Architecture of the Application

© Aptech Limited

Reference Application
Students may refer to https://cit.io/en/transport-analytics/ to understand the general concept of public transport analytics, network monitoring, operational indicators, movement flows, occupancy, delay analysis, stop-level and trip-level analytics, and transport optimization.

The reference platform provides examples of centralized mobility analytics, operational KPI monitoring, occupancy analysis, movement-flow analysis, trip and stop-level performance analysis, and scenario-based optimization. Teams must not copy or reproduce the reference application; they must create their own original application based on the functional requirements defined in this SRS.

Development Phase of the Application

The development phase of the application includes the following steps:

Step 1: Transport Dataset Creation

Teams must create their own large-scale and interconnected public transport dataset.

The dataset should contain:

- Ticketing transactions
- Passenger entries
- Passenger exits
- Routes
- Route stops
- Trips
- Scheduled arrival times
- Actual arrival times
- Scheduled departure times
- Actual departure times
- Vehicle assignments
- Vehicle capacity
- Passenger counts
- Delay records

© Aptech Limited

- Stop information
- Route distance
- Service calendars
- Fare information
- Location information
- GPS or simulated movement data

Each entity must use suitable identifiers such as:

- Passenger ID
- Ticket ID
- Route ID
- Trip ID
- Vehicle ID
- Stop ID
- Service ID

Step 2: Bigdata Storage

The generated datasets must be stored using suitable Big Data formats and technologies.

Teams must demonstrate the use of:

- HDFS
- CSV
- JSON
- Parquet
- Relational or NoSQL storage where required

At least one large analytical dataset must be stored in Parquet format.

Step 3: Data Ingestion

The transport data must be ingested using Apache Spark or PySpark.

© Aptech Limited

Teams must demonstrate:

- Multiple file ingestion
- Explicit schema definition
- Schema inference
- Large dataset loading
- Partitioning
- Data-type validation
- HDFS reading
- Parquet reading

Step 4: Data Quality Analysis

The application must detect:

- Missing ticket records
- Missing route IDs
- Invalid stop IDs
- Duplicate ticket transactions
- Duplicate trips
- Negative passenger counts
- Invalid timestamps
- Impossible arrival times
- Departure before arrival
- Vehicle capacity violations
- Invalid delay values
- Missing vehicle assignments
- Broken stop sequences
- Invalid route distances
- Unknown passengers
- Missing trip records

A Data Quality Report must be generated.

© Aptech Limited

Step 5: Data Cleaning

The application must clean, correct, flag, remove, or quarantine invalid records using documented rules.

Teams must retain a record of:

- Original record
- Data-quality issue
- Cleaning rule
- Corrected value
- Final status

Step 6: Data Integration

Transport tables must be integrated using Spark SQL or PySpark joins.

Required joins should include:

- Tickets with passengers
- Tickets with trips
- Trips with routes
- Trips with vehicles
- Trips with schedules
- Routes with route stops
- Route stops with stops
- Trips with delay records
- Trips with passenger counts
- Stops with location information

Step 7: Feature Engineering

The application must generate analytical features such as:

- Passenger count per trip
- Passenger count per route
- Passenger count per stop
- Boarding count

© Aptech Limited

- Alighting count
- Vehicle occupancy percentage
- Delay duration
- Travel time
- Waiting time
- Route utilization
- Stop utilization
- Peak-hour indicator
- Day-of-week indicator
- Weekend indicator
- Passenger-flow direction
- Route reliability
- Trip punctuality
- Delay frequency
- Schedule deviation
- Capacity utilization
- Headway
- Passenger demand growth
- Historical demand average
- Route load factor

Step 8: Exploratory Data Analysis

The application must identify:

- Highest-demand routes
- Lowest-demand routes
- Busiest stops
- Least-used stops
- Most delayed routes
- Most punctual routes

© Aptech Limited

- Peak hours
- Peak days
- Passenger-flow trends
- High-occupancy trips
- Underutilized trips
- Route-wise travel-time variation
- Stop-level boarding patterns
- Stop-level alighting patterns

Step 9: Passenger Flow Analysis

The application must analyse passenger movement across the transport network.

At minimum, the application must identify:

- Origin stops
- Destination stops
- Boarding patterns
- Alighting patterns
- Route demand
- Direction-wise demand
- Stop-to-stop flows
- High-volume origin-destination pairs

Step 10: Origin Destination Matrix

The application must generate an Origin-Destination matrix. The matrix should show:

- Origin Stop
- Destination Stop
- Passenger Count
- Time Period
- Route
- Day Type

© Aptech Limited

Users should be able to filter by route, period, stop, and service type.

Step 11: Peak Travel Period Analysis

The application must identify:

- Morning peak
- Evening peak
- Midday demand
- Weekend demand
- Holiday demand
- Route-specific peaks
- Stop-specific peaks

Peak periods must be calculated using actual demand rather than static predefined time slots.

Step 12: Real-Time Sound Monitoring

The application must detect overcrowded trips using:

- Passenger count
- Vehicle capacity
- Occupancy percentage
- Duration of overcrowding
- Consecutive overloaded stops

The application should categorize occupancy as:

- Low
- Moderate
- High
- Overcrowded
- Critical

Step 13: Persistent Overcrowding Analysis

A single overloaded trip should not automatically classify a route as overcrowded.

© Aptech Limited

The application must detect persistent overcrowding using:

- Repeated overload
- Time-period consistency
- Route direction
- Stop sequence
- Day-of-week frequency

Step 14: Underutilized Service Detection

The application must identify services with consistently low passenger demand.

Analysis should consider:

- Occupancy
- Number of trips
- Operating frequency
- Route length
- Time period
- Day
- Passenger demand

Step 15: Route Performance Scoring

Each route must receive a composite performance score.

The score should consider:

- Demand
- Occupancy
- Punctuality
- Delay frequency
- Travel time
- Reliability
- Passenger load
- Underutilization
- Overcrowding

© Aptech Limited

Routes should be classified into categories such as:

- High Performing
- High Demand, but Unreliable
- Reliable, but Underutilized
- Overcrowded
- Low Performing

Step 16: Tricky Route Cases

The application must correctly handle scenarios such as:

- High-demand route with poor punctuality
- Low-demand route with excellent punctuality
- Profitable or necessary route with low occupancy
- Route overcrowded only during one direction
- Route overcrowded only during specific stops
- High passenger count caused by one major event
- Delay caused by one abnormal day
- Route performing well on weekdays but poorly on weekends
- Route with high demand but excess frequency
- Route with low demand because of schedule mismatch

Step 17: Delay Analysis

The application must analyze delays by:

- Route
- Trip
- Stop
- Vehicle
- Time
- Day
- Direction
- Distance travelled

© Aptech Limited

Step 18: Delay Pattern Detection

The application must identify recurring delay patterns.

Examples include:

- Morning congestion delays
- Evening congestion delays
- Delay accumulation across stops
- Specific stop bottlenecks
- Certain vehicles consistently delayed
- Specific routes delayed on particular days

Step 19: Delay Prediction

The application should predict delay risk for future trips.

Possible features include:

- Route
- Time of day
- Day of week
- Historical delay
- Passenger load
- Number of stops
- Travel distance
- Historical travel time
- Peak period
- Vehicle

At least three suitable models must be evaluated.

Step 20: Delay Severity Classification

Predicted delays may be classified as:

- On Time
- Minor Delay
- Moderate Delay

© Aptech Limited

- Major Delay
- Severe Delay

The thresholds must be configurable.

Step 21: Route Clustering

Routes must be grouped using unsupervised Data Science techniques.

Possible clustering attributes include:

- Demand
- Average occupancy
- Average delay
- Reliability
- Trip frequency
- Peak demand
- Travel time
- Number of stops

Teams may use:

- K-Means
- Hierarchical Clustering
- DBSCAN
- Other suitable methods

Step 22: Passenger Demand Forecasting

The application must forecast passenger demand for:

- Routes
- Stops
- Time periods
- Days
- Trips

Forecasting should support configurable future periods.

© Aptech Limited

Step 23: Time-Aware Validation

Forecasting models must use chronological training and testing.

Future records must not leak into historical training data.

Step 24: Forecast Evaluation

Forecasting models should be evaluated using:

- MAE
- RMSE
- MAPE
- R^2 where appropriate

The selected model must outperform a documented simple baseline.

Step 25: Occupancy Forecasting

The application should predict future occupancy for selected routes or trips.

Inputs may include:

- Historical passenger count
- Trip time
- Route
- Day
- Service frequency
- Previous occupancy
- Peak indicator

Step 26: Crowding Risk Prediction

The application should estimate the probability that a trip will exceed a configured occupancy threshold.

High-risk trips should be flagged.

Step 27: Stop-Level Performance Analysis

Each stop should be analyzed using:

- Boarding count
- Alighting count

© Aptech Limited

- Passenger turnover
- Average delay
- Demand by time
- Trip frequency
- Route connectivity

Step 28: Bottleneck Stop Detection

The application must identify stops associated with:

- Repeated delays
- Long dwell times
- High boarding volumes
- High passenger accumulation
- Frequent overcrowding

Step 29: Travel-Time Analysis

The application must compare:

- Scheduled travel time
- Actual travel time
- Historical average
- Peak travel time
- Off-peak travel time

Step 30: Route Reliability Analysis

Route reliability should consider:

- Percentage of on-time trips
- Delay variation
- Missed schedules
- Excessively early arrivals
- Travel-time consistency

© Aptech Limited

Step 31: Schedule Adherence Analysis

The application must compare scheduled and actual movement.

The application should identify:

- Early arrivals
- On-time arrivals
- Late arrivals
- Missed trips
- Irregular service intervals

Step 32: Headway Analysis

The application should calculate the time difference between consecutive vehicles on the same route.

It must identify:

- Bunching
- Excessive gaps
- Irregular headways

Step 33: Bus or Vehicle Bunching Detection

The application should detect when two or more vehicles on the same route operate unusually close together.

Such events should be flagged for investigation.

Step 34: Service Frequency Analysis

The application must compare:

- Scheduled frequency
- Passenger demand
- Occupancy
- Peak demand
- Delay

© Aptech Limited

The goal is to identify services with:

- Too much frequency
- Too little frequency
- Well-matched frequency

Step 35: Schedule Improvement Recommendations

The application should recommend actions such as:

- Increase frequency during peak periods
- Reduce frequency during consistently underutilized periods
- Add additional trips
- Shift departure time
- Extend dwell time
- Reduce dwell time
- Adjust route timing
- Review stop sequence

Recommendations must be evidence-based.

Step 36: Demand-Supply Gap Analysis

The application must compare passenger demand with available transport capacity.

It should identify:

- Excess demand
- Excess supply
- Balanced services

Step 37: Capacity Optimization

The application should identify cases where:

- Larger vehicles are required
- Smaller vehicles may be sufficient
- Additional trips are required
- Existing frequency may be reduced

© Aptech Limited

Step 38: Special Event Detection

The application should detect unusual spikes caused by:

- Events
- Festivals
- Sports matches
- Weather events
- Simulated disruptions

These should not automatically redefine normal demand.

Step 39: Anomaly Detection

The application must detect unusual transport behavior such as:

- Sudden passenger spikes
- Sudden demand drops
- Abnormal delays
- Unexpected route usage
- Excessively low passenger counts
- Duplicate ticketing
- Impossible occupancy
- Abnormal travel time
- Irregular stop activity

Step 40: Passenger Behavior Segmentation

Passenger usage patterns may be segmented based on:

- Travel frequency
- Preferred routes
- Time-of-day behavior
- Weekday/weekend travel
- Trip distance
- Origin-destination patterns

© Aptech Limited

Possible segments include:

- Daily Commuters
- Occasional Travellers
- Peak-Hour Travellers
- Weekend Travellers
- Long-Distance Travellers

Step 41: Big Data Model Development

Teams must develop models using Spark MLlib.

At least three suitable algorithms must be evaluated for applicable predictive tasks.

Examples include:

- Logistic Regression
- Decision Tree
- Random Forest
- Gradient-Boosted Trees
- Linear Regression
- K-Means

Step 42: Independent Python Data Science Pipeline

Teams must develop an independent Python Data Science pipeline.

It may use:

- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Statsmodels
- SciPy

Spark-generated predictions must not be reused as Python predictions.

© Aptech Limited

Step 43: Dual-Pipeline Comparison

At least one major analytical task must be solved independently using both:

- Spark MLlib
- Python Data Science

Recommended comparison tasks include:

- Delay prediction
- Demand prediction
- Route classification
- Occupancy-risk prediction

Step 44: Model Comparison Report

The comparison must include:

- Trip ID or Route ID
- Actual result
- Spark result
- Python result
- Match or mismatch
- Probability or predicted value
- Difference
- Explanation of disagreement
- Overall consistency rate

At least 100 unseen analytical cases must be compared.

Step 45: Recommendation Engine

The application must generate evidence-based operational recommendations.

Examples include:

- Increase frequency on overloaded route
- Reduce underutilized trips
- Modify schedule around recurrent delays
- Allocate higher-capacity vehicle

© Aptech Limited

- Investigate bottleneck stop
- Adjust departure time
- Improve spacing between vehicles

Step 46: Recommendation Evidence

Every recommendation must show supporting evidence.

Example:

Recommended Action:

Increase Route R12 frequency between 08:00 and 09:00.

Reason:

Average occupancy: 94%

Critical occupancy events: 18

Passenger demand growth: 21%

Average headway: 17 minutes

Repeated overload observed on weekdays

Step 47: Recommendation Priority

Recommendations should be prioritized as:

- Low
- Medium
- High
- Critical

Priority should reflect passenger impact and operational severity.

Step 48: What-If Scenario Analysis

Users must be able to simulate changes such as:

- Increase route frequency
- Decrease route frequency
- Add a vehicle
- Change vehicle capacity
- Shift trip start time

© Aptech Limited

- Remove a low-demand trip
- Add a new stop
- Increase predicted demand

Step 49: Scenario Impact Analysis

The application should estimate the impact on:

- Occupancy
- Passenger load
- Waiting time
- Route capacity
- Demand coverage
- Overcrowding risk

All simulated results must be clearly identified as estimates.

Step 50: Executive Dashboard

The dashboard should display:

- Total passengers
- Total trips
- Average occupancy
- Average delay
- On-time percentage
- High-demand routes
- Overcrowded routes
- Underutilized routes
- Critical alerts

Step 51: Passenger Flow Dashboard

The dashboard should display:

- Boarding
- Alighting
- Origin-destination flows

© Aptech Limited

- Peak periods
- Route demand
- Stop demand

Step 52: Route Performance Dashboard

The dashboard should display:

- Route ranking
- Route performance score
- Delay
- Occupancy
- Reliability
- Passenger demand

Step 53: Delay Dashboard

The application should display:

- Route-wise delay
- Stop-wise delay
- Delay trends
- Delay severity
- Delay prediction

Step 54: Occupancy Dashboard

The application should display:

- Current or historical occupancy
- Overcrowded routes
- High-risk trips
- Occupancy trends
- Capacity utilization

© Aptech Limited

Step 55: Forecast Dashboard

The dashboard should display:

- Historical demand
- Forecast demand
- Actual versus predicted
- Forecast error
- Future high-demand periods

Step 56: Route Map Visualization

The Web interface should provide a route or network map.

The map may display:

- Stops
- Routes
- High-demand locations
- Overcrowding
- Delay hotspots
- Passenger flows

Step 57: Search and Filtering

Users should be able to filter by:

- Date
- Route
- Trip
- Stop
- Vehicle
- Direction
- Peak/off-peak
- Delay level
- Occupancy level

© Aptech Limited

Step 58: Reports and Export

The application should generate downloadable reports for:

- Passenger demand
- Route performance
- Delays
- Occupancy
- Stop performance
- Forecasting
- Route clustering
- Recommendations
- Spark versus Python comparison

Hint

For this project, teams are expected to create their own large and realistic public transport dataset. A complete competition-ready dataset will not be provided.

The dataset must contain a minimum of:

- **At least 2,000,000 ticketing or passenger movement records**
- **At least 500,000 trip-level passenger records**
- **At least 100 routes**
- **At least 500 stops**
- **At least 250 vehicles**
- **At least 50,000 unique passengers**
- **At least 12 months of historical operation**
- **At least 250,000 delay records**
- Multiple service calendars and schedules

The dataset must contain connected tables such as:

- Passengers
- Tickets
- Routes

© Aptech Limited

- Stops
- Route_Stops
- Trips
- Schedules
- Vehicles
- Passenger_Counts
- Delays
- GPS_Events
- Service_Calendar

The generated data must include difficult and realistic conditions such as:

- Missing values
- Duplicate tickets
- Invalid timestamps
- Trip cancellations
- Early arrivals
- Delays
- Vehicle changes
- Seasonal demand
- Weekday/weekend differences
- Peak-hour demand
- Route direction differences
- Special events
- Overcrowded services
- Low-demand services
- Irregular headways
- Vehicle bunching
- Stop bottlenecks
- Passenger spikes

© Aptech Limited

- New routes
- New stops
- New service schedules

Teams must submit the data-generation scripts.

Downloading a ready-made transport dataset without significant transformation, integration, and augmentation will not satisfy the requirement.

Predictive models must use appropriate training, validation, and testing sets.

Time-dependent modelling must use chronological validation to avoid future-data leakage.



1.3 Purpose of the Document

The purpose of this document is to outline the design, functionality, and implementation plan of the **UrbanTransit IQ** application. This document ensures a shared understanding of the project objectives and provides a structured roadmap for the development, testing, and deployment of the application. It is intended for project stakeholders, developers, evaluators, public transport authorities, transport operators, data analysts, and administrators.

© Aptech Limited

1.4 Scope of Project

The scope of this project is to design and implement a Big Data and Data Science-powered, Web-based **UrbanTransit IQ** application. The application will assist transport authorities, public transport operators, analysts, and administrators in analyzing large volumes of ticketing, route, passenger-count, timing, stop, schedule, and delay data.

The application will identify peak travel periods, overcrowded routes, low-performing services, delay patterns, passenger-flow trends, stop-level demand, route bottlenecks, and opportunities for schedule improvement. It will also perform passenger-demand forecasting, occupancy prediction, route clustering, delay prediction, anomaly detection, and dual-pipeline result comparison using Apache Spark and an independent Python Data Science workflow.

Integration with live commercial ticketing systems, traffic-control systems, GPS hardware, payment gateways, or operational vehicle-control platforms falls outside the mandatory scope of this project.



© Aptech Limited

1.5 Constraints

The **UrbanTransit IQ** project faces certain constraints such as dependency on quality, completeness, consistency, and accuracy of ticketing, route, schedule, passenger-count, timing, stop, and delay datasets.

The application's effectiveness is limited by the size, realism, diversity, and reliability of the generated dataset, as well as the computational resources available for Big Data processing. Differences may also occur between analytical results generated by the Spark/PySpark pipeline and the independent Python Data Science pipeline.

Demand and delay predictions may also be affected by unpredictable conditions such as traffic incidents, weather, special events, route disruptions, and unusual passenger behavior.

Privacy and security concerns regarding the storage and handling of passenger, ticketing, location, and transport-operation information must also be addressed in accordance with applicable data protection regulations.

1.6 Functional Requirements

Creating **UrbanTransit IQ** involves several steps and components. Following are the functional requirements:

i. **User Authentication** - The application should provide secure access for transport analysts, administrators, and authorized users.



ii. **Role-Based Access Control –** Different access permissions should be provided to administrators, operators, analysts, and evaluators.

iii. **Route Management –** The application should store route information and route status.

iv. **Stop Management –** Stop information and stop sequence should be maintained.

v. **Vehicle Management –** Vehicle ID, type, capacity, and operational status should be stored.

© Aptech Limited

vi. **Trip Management –** Scheduled and completed transport trips should be maintained.



vii. **Ticketing Data Management –** Passenger transaction data should be stored and processed.

viii. **Passenger Count Management –** Boarding, alighting, and occupancy values should be maintained.

ix. **Schedule Management –** Scheduled arrival and departure information should be stored.

x. **Delay Management –** Actual delay records should be maintained.

xi. **Bigdata Ingestion –** Spark/PySpark should process large transport datasets.



xii. **HDFS Storage –** Large datasets should be stored using Hadoop-compatible storage.

xiii. **Schema Validation –** Input schemas and data types should be validated.

xiv. **Data Quality Analysis –** Missing, duplicate, inconsistent, and invalid records should be identified.

xv. **Data Cleaning –** Invalid records should be corrected or flagged using documented rules.

xvi. **Spark SQL Processing –** Spark SQL should support large-scale queries and aggregation.

xvii. **Data Partitioning –** Large datasets should be appropriately partitioned.



xviii. **Parquet Storage –** Processed Big Data should use Parquet wherever suitable.

xix. **Feature Engineering –** Derived Data Science features should be generated.

© Aptech Limited

xx. **Passenger Flow Analysis –** Passenger movement across stops and routes should be analyzed.

xxi. **Origin-Destination Analysis –** Origin-destination passenger flows should be calculated.



xxii. **Peak Travel Detection –** Peak travel periods should be identified.

xxiii. **Overcrowding Detection –** High-occupancy and overcrowded trips should be detected.

xxiv. **Persistent Overcrowding Analysis –** Repeated overload should be distinguished from isolated events.

xxv. **Underutilization Detection –** Low-demand services should be identified.

xxvi. **Route Performance Scoring –** Routes should receive multi-factor performance scores.



xxvii. **Route Classification –** Routes should be classified into operational categories.

xxviii. **Delay Analysis –** Delay patterns should be analyzed across routes, trips, and stops.

xxix. **Delay Prediction –** Future delay risk should be predicted.



xxx. **Delay Severity Classification –** Predicted and historical delays should be categorized.

xxxi. **Passenger Demand Forecasting –** The application should forecast future passenger demand.

xxxii. **Occupancy Forecasting –** Future occupancy should be estimated.

xxxiii. **Crowding Risk Prediction –** High-risk overcrowding trips should be predicted.

xxxiv. **Route Clustering –** Routes should be grouped according to operational similarities.

© Aptech Limited



xxxv. **Stop Performance Analysis –** Stop-level usage and performance should be analyzed.

xxxvi. **Bottleneck Stop Detection –** High-delay or high-load stops should be identified.

xxxvii. **Travel-Time Analysis –** Scheduled and actual travel times should be compared.

xxxviii. **Route Reliability Analysis –** Route punctuality and consistency should be measured.

xxxix. **Schedule Adherence Analysis –** Schedule deviations should be detected.

xl. **Headway Analysis –** Vehicle spacing should be measured.



xli. **Vehicle Bunching detection –** Closely grouped vehicles should be identified.

xlii. **Service Frequency Analysis –** Demand should be compared with service frequency.

xliii. **Demand Supply Gap Analysis –** Excess demand and excess capacity should be identified.



xliv. **Capacity Optimization –** Vehicle capacity and trip frequency recommendations should be generated.

xlv. **Special Event Detection –** Unusual demand spikes should be separated from normal patterns.

xlvi. **Anomaly detection –** Normal passenger, delay, route, stop, and trip behavior should be detected.

xlvii. **Passenger Segmentation –** Passenger usage patterns should be grouped.

xlviii. **Spark MLib Modelling –** At least three suitable Spark models should be evaluated.

© Aptech Limited



xlix. **Independent Python Data Science Modelling** - Separate Python models should be developed.

l. **Dual Pipeline Comparison –** Selected Spark and Python model outputs should be compared.

li. **Consistency Analysis –** Agreement and disagreement should be quantified.

lii. **Recommendation Engine –** Evidence-based operational recommendations should be generated.



liii. **Schedule Optimization Recommendation –** Timing and frequency improvements should be suggested.

liv. **Capacity Recommendations –** Capacity changes should be recommended.

lv. **What-If Analysis –** Operational scenarios should be simulated.

lvi. **Scenario Impact Analysis –** Simulated changes should show likely operational effects.

lvii. **Executive Dashboard –** Network-level KPIs should be displayed.

lviii. **Passenger Flow Dashboard –** Passenger movement should be displayed.

lix. **Route Performance Dashboard –** Route performance should be displayed.

lx. **Delay Dashboard –** Delay trends and predictions should be displayed.



lxi. **Occupancy Dashboard –** Occupancy and crowding risk should be displayed.

lxii. **Forecast Dashboard –** Historical and forecast demand should be displayed.

lxiii. **Route Map Visualization –** Transport routes and network hotspots should be visualized.

© Aptech Limited

lxiv. **Search and Filtering –** Analytical results should support multiple filtering options.

lxv. **Downloadable Reports –** Reports should be generated.

lxvi. **Data Export –** Analytical datasets should be exportable.



lxvii. **Database Storage –** Application metadata and analytical results should be stored securely.

lxviii. **Model Version Tracking –** Model versions should be maintained.

lxix. **Audit Trail –** Processing and administrative actions should be logged.

lxx. **Error Handling –** Processing, modelling, Spark, and database errors should be displayed clearly.



lxxi. **Spark Job Monitoring –** Processing job status should be displayed.

lxxii. **Responsive Web Interface –** The application should provide an intuitive Web interface.

The application must provide analytical conclusions and operational recommendations and not simply charts and summary statistics.



© Aptech Limited

1.7 Non-Functional Requirements

There are several non-functional requirements that should be fulfilled by the application.

1. **Performance:** The application should efficiently process at least two million passenger or ticketing records. Standard analytical dashboards should return within five seconds after processed datasets have been prepared.
2. **Scalable:** The architecture should be capable of scaling to at least ten million passenger movement records without requiring complete redesign.
3. **Usable:** The application should provide an intuitive interface for transport operators, authorities, analysts, and administrators.
4. **Accuracy:** Classification models should target at least 85% test accuracy or macro F1-score of at least 0.80 where appropriate. Forecasting models should demonstrate measurable improvement over a documented baseline.
5. **Available:** The application should remain operational during competition evaluation with at least 99% uptime under normal conditions.



These are the bare minimum expectations from the project. It is a must to implement the FUNCTIONAL and NON-FUNCTIONAL requirements given in this SRS.

Once they are complete, you can use your own creativity and imagination to add more features if required.

1.8 Competition Integrity and Anti-Shortcut Requirements

To ensure genuine technical work during the five-day competition, the following are mandatory:

1. Meaningful GitHub commits must be made across all five competition days.
2. Every team member must contribute code, analytics, documentation, testing, or modelling work.

© Aptech Limited

3. Teams must maintain a Development Log documenting:

    - Work completed
    - Dataset changes
    - Data-quality problems
    - Spark failures
    - Model errors
    - Modifications made
    - Testing completed
    - Performance improvements

    Evaluators may select any of the following and ask the team to explain it.

    - Spark transformation
    - Spark SQL query
    - Python function
    - Feature
    - Model
    - Forecast
    - Recommendation

4. Teams must complete surprise modifications such as:

    - Add a new transport route
    - Add a new stop
    - Change vehicle capacity
    - Modify route thresholds
    - Change delay thresholds
    - Change forecast period
    - Add a new KPI
    - Modify a Spark transformation
    - Add an anomaly rule
    - Add a dashboard filter

5. Evaluators may deliberately introduce a defect that the team must diagnose and fix.

© Aptech Limited

6. Hidden transport data will be provided during final evaluation.
7. Hidden data may contain:
    - Missing routes
    - New stops
    - New schedules
    - Duplicate tickets
    - Unknown vehicles
    - Delay spikes
    - New passenger patterns
    - Unexpected overcrowding
    - Unseen passenger flows
    - Data-quality defects
8. Teams must demonstrate that Spark and Python models independently generate their outputs.
9. Spark-generated predictions must not be copied into the Python pipeline.
10. Teams may be asked to explain why Spark and Python disagree.
11. Hard-coded predictions, fabricated accuracy, manually created dashboard values, or hidden decision APIs may lead to disqualification.
12. Any AI tool used must be declared in `AI_USAGE.md`.
13. AI-generated code must be independently reviewed, modified where necessary, tested, and understood by the team.
14. The final demand predictions, delay predictions, classifications, and recommendations must not be generated by external generative-AI APIs.
15. Failure to explain submitted code may result in reduced or zero marks for the relevant module.

© Aptech Limited

1.9 Interface Requirements

1.9.1 Hardware

Intel Core i5/i7 Processor or higher
8 GB RAM or higher
Color SVGA monitor
500 GB Hard Disk space
Mouse and Keyboard

1.9.2 Software

Technologies to be used:

1. **Frontend**: HTML5, CSS3, JavaScript, Bootstrap, Streamlit, React, or another suitable frontend technology
2. **Backend**: Flask, Django, FastAPI, or Streamlit
3. **Big Data Eco System**: Apache Hadoop ecosystem, HDFS, Apache Spark, PySpark, Spark SQL
4. **Big Data Storage**: HDFS, CSV, JSON, Parquet, or Hadoop-compatible storage
5. **Database**: MongoDB, PostgreSQL, MySQL, Firebase, SQLite, or another suitable database
6. **Programming/IDE**: Python, Jupyter Notebook, Anaconda, Google Colab, PyCharm, or Visual Studio Code
7. **Data Science and Analytics**: Apache Spark MLlib, Scikit-learn, XGBoost, Statsmodels, SciPy, or other suitable Data Science libraries
8. **Data Science Libraries**: NumPy, Pandas, Matplotlib, Plotly, Joblib, or other suitable libraries
9. **Big Data Query and Analysis**: Spark SQL, PySpark DataFrame API, SQL
10. **Visualization**: Plotly, Matplotlib, Streamlit charts, Power BI, Tableau, or suitable Web visualization technologies
11. **Mapping**: Folium, Plotly Maps, Leaflet, or other suitable mapping libraries
12. **Testing**: pytest, unittest, Postman, Spark testing utilities, or other suitable tools
13. **Version Control**: Git and GitHub
14. **Deployment**: Render, Railway, PythonAnywhere, Streamlit Community Cloud, or another suitable hosting platform

© Aptech Limited



1.10 Project Deliverables

Each team must design, build, test, document, deploy, and demonstrate the complete application of **UrbanTransit IQ**.

1. Project Report

The report must include:

- Problem definition
- Background and necessity
- Proposed solution
- Purpose
- Scope
- Assumptions
- Constraints
- Functional requirements
- Non-functional requirements
- Application architecture
- Big Data architecture
- Database design
- Data dictionary

© Aptech Limited

- Entity relationships
- Data Flow Diagram
- Use Case Diagram
- Activity Diagram
- Sequence Diagram
- Dataset-generation methodology
- Data-quality methodology
- Spark pipeline
- Spark SQL queries
- Feature engineering
- Passenger-flow analysis
- Origin-destination analysis
- Route-performance methodology
- Delay analysis
- Delay prediction
- Route clustering
- Passenger-demand forecasting
- Occupancy prediction
- Anomaly detection
- Spark MLlib model
- Python Data Science model
- Dual-pipeline comparison
- Recommendation engine
- Testing
- Security
- Privacy
- Limitations
- Future enhancement

2. Source Code

The public GitHub repository should include:

```
README.md

AI_USAGE.md

requirements.txt

LICENSE

src/
```

© Aptech Limited

```
templates/
static/
data_generator/
raw_data/
processed_data/
parquet_data/
hdfs_scripts/
spark_jobs/
spark_sql/
python_pipeline/
notebooks/
models/
forecasting/
route_clustering/
delay_analysis/
occupancy_analysis/
recommendation_engine/
database/
tests/
sample_data/
documentation/
screenshots/
reports/
config/
```

3. Big Data Dataset

Teams must submit:

- Data-generation scripts
- Data dictionary
- Dataset schemas

© Aptech Limited



- Primary/foreign key definitions
- Raw sample data
- Cleaned data
- Parquet data
- Training data
- Validation data
- Test data
- Dataset statistics

4. Spark Processing Evidence

Teams must submit:

- HDFS ingestion
- Spark ingestion scripts
- Schemas
- Cleaning transformations
- Spark joins
- Spark SQL queries
- Aggregations
- Partition strategy
- Parquet output
- Processing logs
- Performance results

5. Spark MLib Evidence

The submission must include:

- Features
- Algorithms evaluated
- Hyperparameters
- Training results
- Validation results
- Test results
- Metrics

© Aptech Limited

- Confusion matrix where applicable
- Saved model
- Model version
- Sample predictions

At least three suitable models must be evaluated.

6. Python Data Science Model Evidence

Teams must submit:

- Python preprocessing
- Feature engineering
- Algorithms evaluated
- Hyperparameters
- Training results
- Validation results
- Test results
- Accuracy
- Precision
- Recall
- F1-score
- Forecast metrics
- Saved model
- Sample predictions

7. Dual Pipeline Comparison Report

The report must compare at least 100 unseen cases.

It must include:

- Trip/Route ID
- Actual result
- Spark result
- Python result

© Aptech Limited



- Match/mismatch
- Prediction probability or value
- Numerical difference
- Consistency status
- Explanation of significant disagreement
- Overall agreement percentage

8. Transport Intelligence Report

The report must contain:

- Busiest routes
- Low-performing routes
- Overcrowded routes
- Underutilized routes
- Peak periods
- Most delayed routes
- Most reliable routes
- Passenger-flow patterns
- Bottleneck stops
- Demand forecasts
- Crowding risk
- Route clusters
- Service gaps
- Schedule recommendations

9. Test Cases

Tests must include:

- Functional tests
- Integration tests
- Big Data ingestion tests
- Schema tests
- Data-quality tests
- Spark tests
- Spark SQL tests
- Model tests
- Forecast tests
- Delay tests

© Aptech Limited



- Occupancy tests
- Route tests
- Stop tests
- Dual-pipeline tests
- Anomaly tests
- Hidden-data readiness tests
- Security tests
- Boundary tests

10. Installation Instructions

Documentation must explain:

- Python installation
- Java installation
- Hadoop installation
- HDFS configuration
- Apache Spark installation
- PySpark configuration
- Virtual environment setup
- Database setup
- Dataset generation
- Spark execution
- Model execution
- Web application execution
- Test execution
- Troubleshooting

11. Execution Instructions

The README must explain:

- Login
- Dataset generation
- Data ingestion
- HDFS storage

© Aptech Limited

- Spark processing
- Spark SQL queries
- Passenger-flow analysis
- Route analysis
- Delay analysis
- Forecasting
- Occupancy prediction
- Route clustering
- Spark model execution
- Python model execution
- Model comparison
- Recommendation generation
- Dashboard usage
- Report export

12. GitHub Repository

The repository must:

- Be public
- Contain meaningful commits from all team members
- Contain complete source code
- Include Big Data scripts
- Include Python Data Science scripts
- Include dataset-generation scripts
- Include model files
- Include sample data
- Include testing
- Include screenshots
- Include evaluator credentials
- Include documentation

© Aptech Limited

- Include blog link
- Include video link

13. Deployed Application

Teams should submit:

- Public application URL
- Evaluator credentials
- Administrator credentials
- Sample data
- Testing instructions

14. Demonstration Video

A **.mp4** demonstration video is mandatory.

The video must demonstrate:

- Login
- Dataset generation
- HDFS or Big Data ingestion
- Spark processing
- Data-quality analysis
- Passenger-flow analysis
- Origin-destination analysis
- Peak-period detection
- Overcrowding detection
- Underutilization analysis
- Route-performance analysis
- Delay analysis
- Delay prediction
- Route clustering
- Demand forecasting
- Occupancy forecasting

© Aptech Limited

- Spark model
- Python model
- Dual-pipeline comparison
- Recommendation engine
- What-if analysis
- Network map
- Dashboard
- Report generation

15. Technical Blog

Each team must publish a technical blog of at least 2,000 words.

The blog must discuss:

- Transport problem
- Big Data architecture
- Dataset creation
- Data quality
- Hadoop
- HDFS
- Apache Spark
- PySpark
- Spark SQL
- Feature engineering
- Passenger flow
- Origin-destination analysis
- Route performance
- Delay analytics
- Forecasting
- Occupancy
- Route clustering

© Aptech Limited

- Spark MLlib
- Python Data Science
- Dual-pipeline comparison
- Challenges
- Testing
- Performance
- Security
- Lessons learned
- Limitations
- Future enhancements

16. AI Tool Usage Declaration

Teams must not directly submit AI-generated code or content without independently reviewing, modifying, testing, and understanding it.

All AI tools used must be declared in AI_USAGE.md.

The declaration must include:

- Tool name
- Purpose
- Type of help
- Files affected
- Modifications made
- Testing completed
- Verifying team member

AI-generated images may be used for visual representation if declared.

Final transport analytics, predictions, classifications, forecasts, and recommendations must be produced by the team's own Big Data, Data Science, Machine Learning, and application logic.

© Aptech Limited

17. Final Submission Checklist

The final submission must include:

- Project report
- Public GitHub URL
- Complete source code
- Data-generation scripts
- Big Data dataset
- Data dictionary
- HDFS scripts
- Spark jobs
- Spark SQL files
- Parquet data
- Spark MLlib models
- Python Data Science models
- Dual-pipeline comparison report
- Transport intelligence report
- Installation instructions
- Execution instructions
- Deployment URL
- Demonstration video
- Technical blog
- `AI_USAGE.md`
- Team contribution record

Documentation is a critical part of the project. All significant aspects of the Big Data architecture, dataset, Spark pipeline, Data Science models, passenger-flow analysis, forecasting, route analytics, testing, deployment, security, and limitations must be documented completely.

~~~ End of Document ~~~

© Aptech Limited